

Redis SQL Trino

Version 0.4.2

Table of Contents

Introduction.....	1
Installation.....	2
Configuration.....	3
SQL support	8
Clients	13
Build.....	14
Complete Walkthrough.....	15

Introduction

Redis SQL Trino is a [Trino](#) connector which allows access to Redisearch data from Trino.

This guide provides documentation and usage information across the following topics:

- [Installation](#)
- [Configuration](#)
- [SQL support](#)
- [Clients](#)
- [Walkthrough](#)

Installation

Trino

Trino installation instructions are available at <https://trino.io/docs/current/installation.html>.

RediSearch connector

Download [latest release](#) and unzip without any directory structure under:

`<trino>/plugin/redisearch`

Create a RediSearch connector configuration file and change/add [properties](#) as needed.

Configuration

To configure the Redisearch connector, create a catalog properties file and change/add properties as needed.

etc/catalog/redisearch.properties

```
connector.name=redisearch
redisearch.uri=redis://localhost:6379
```

Table 1. Connector properties

Property name	Description	Default
<code>redisearch.default-schema-name</code>	The schema that contains all tables defined without a qualifying schema name.	<code>default</code>
<code>redisearch.case-insensitive-names</code>	Match index names case insensitively.	<code>false</code>
<code>redisearch.cursor-count</code>	Number of rows read during each aggregation cursor fetch.	<code>1000</code>
<code>redisearch.narrow-scan-cursor-count</code>	Optional batch size for unbounded document scans of at most two input fields. Maximum 10000; 0 uses <code>cursor-count</code> . Applies to the initial request and later cursor reads. Larger batches increase response/hydration memory; only one cursor read is prefetched.	<code>0</code>
<code>redisearch.aggregation-group-limit</code>	Conservative GROUP BY budget; must not exceed the server <code>MAX_AGGREGATE_GROUPS</code> . Unknown or over-budget document counts retain Trino aggregation. Global aggregates remain eligible.	<code>1000000</code>
<code>redisearch.scan-connections</code>	Number of Redis connections each Trino node's scans take turns on, so that one scan's batches don't hold up other queries' commands.	<code>8</code>

Property name	Description	Default
<code>redisearch.scan-splits</code>	<code>0</code> selects splits automatically from size, estimated CPU capacity and partition balance. <code>1</code> forces a single split; <code>2</code> to <code>64</code> explicitly bounds parallel scans with configured cut points. The connection pool also caps the count. Global operations remain single-split.	<code>0</code>
<code>redisearch.scan-partition-field</code>	Optional indexed NUMERIC field alias used to partition scans. Automatic mode can discover a HASH field when unset. Missing, NOINDEX, unsupported or unsafe field names fall back to one split.	Unset
<code>redisearch.scan-partition-boundaries</code>	Optional comma-separated strictly increasing finite numeric cut points; automatic mode can discover them for HASH indexes. Prefer a stable, evenly distributed field. Missing values stay in the first partition; JSON arrays spanning cut points appear once.	Empty
<code>redisearch.aggregation-pushdown.enabled</code>	Compute supported aggregations in Redis. Set to <code>false</code> to compute them in Trino from cursor batches when large aggregations interfere with latency-sensitive queries. This transfers more rows and can make aggregations substantially slower. Filters still push down.	<code>true</code>

Property name	Description	Default
<code>redisearch.table-cache-refresh</code>	Seconds after which the connector describes a cached index again (<code>FT.INFO</code> , its saved column types and a few documents), in the background: queries keep planning with the cached schema meanwhile. An index no query reads for ten times as long is dropped from the cache, and the next query waits for it. Fields added and indexes dropped outside Trino can take this long to show. <code>0</code> disables the cache.	<code>60</code>
<code>redisearch.dynamic-filtering.enabled</code>	Add the join keys that the other side of a join collects (its dynamic filter) to the query that scans an index.	<code>true</code>
<code>redisearch.dynamic-filtering.wait-timeout</code>	How long a scan waits for the other side of a join to collect its dynamic filter before it starts without it.	<code>20s</code>

Table 2. Redis connection properties

Property name	Description	Default
<code>redisearch.uri</code>	A Redis connection string. Redis URI syntax .	
<code>redisearch.username</code>	Redis connection username.	
<code>redisearch.password</code>	Redis connection password.	
<code>redisearch.cluster</code>	Connect to a Redis Cluster.	<code>false</code>
<code>redisearch.resp2</code>	Force Redis protocol version to RESP2. Trino then computes the aggregations whose results or <code>GROUP BY</code> keys are <code>DOUBLE</code> or <code>REAL</code> : see Pushdown .	<code>false</code>

The RedisSearch connector provides additional security options to support Redis servers with TLS mode.

Table 3. TLS properties

Property name	Description	Default
<code>redisearch.insecure</code>	Allow insecure connections (e.g. invalid certificates) when using SSL.	<code>false</code>
<code>redisearch.cacert-path</code>	X.509 CA certificate file to verify with.	
<code>redisearch.key-path</code>	PKCS#8 private key file to authenticate with (PEM format).	
<code>redisearch.key-password</code>	Password of the private key file, or null if it's not password-protected.	
<code>redisearch.cert-path</code>	X.509 certificate chain file to authenticate with (PEM format).	

Parallel document scans

The connector uses Lettuce for Redis communication. Lettuce supports multiple CPU cores through Netty I/O event loops and computation thread pools. Its default pool sizes follow the available CPU count, with a minimum of two threads, unless configured otherwise. See [Lettuce client resources](#) for the threading settings. A physical connection processes its network events on one event loop; additional client threads do not automatically partition a query or create more Trino sources.

Layer	Work it parallelizes
Lettuce	Redis network I/O and protocol decoding across connections.
Trino connector	Independent scan splits, each with a cursor, processed by separate Trino drivers/workers.
Redis Query Engine	Server-side query computation through query workers.

The connector's existing connection pool supports concurrency between scans. Multiple splits let a single document scan use several connections and Trino source pipelines. Tune Redis query workers independently and measure the combined configuration; client-side parallelism alone does not determine server-side query concurrency.

Automatic split selection is the default (`redisearch.scan-splits=0`). The planner targets at least 250,000 matching rows per split and selects the largest count allowed by the estimated CPU capacity (active Trino workers times coordinator CPU count), the connection pool (eight by default), and a hard ceiling of 64. Small tables and selective filters use one split. A filter's match count is checked before choosing parallelism. For HASH indexes, the planner examines up to three indexed NUMERIC fields, obtains their MIN/MAX in one bounded statistics query, and checks the counts of the proposed ranges. A field is rejected if one partition would contain at least 80% of the matching documents; the planner then tries another candidate. Discovery results, including unsupported cases, are cached for five minutes (up to 256 entries). Statistics queries have a ten-second server timeout and count probes have a one-second timeout; unavailable or partial statistics fall back to one split. The first scan can therefore pay a discovery cost; subsequent scans reuse its plan.

MIN/MAX discovery is a server-side aggregate and can delay concurrent point traffic when Redis query workers are disabled; configured boundaries avoid this discovery work. This is a cost heuristic, not a guarantee of the optimal count under every load. Use the benchmark and explicit overrides when tuning a shared deployment. JSON arrays support explicit partition fields and cut points; automatic MIN/MAX discovery currently applies to HASH indexes.

Set `redisearch.scan-splits=1` to force one source. To override automatic selection and supply known boundaries:

```
redisearch.scan-splits=4
redisearch.scan-connections=4
redisearch.scan-partition-field=ordinal
redisearch.scan-partition-boundaries=1250000,2500000,3750000
```

This partitions one document scan into four Trino sources with independent cursors and distinct scan connection slots on each worker. The NUMERIC field must already be indexed; the connector does not alter the index. Explicit boundaries skip discovery. Choose cut points from the field's distribution; uneven partitions still return complete results but can limit the speedup. If fewer splits are allowed, the connector selects evenly spaced cut points from the configured list and merges the other intervals. The lowest partition includes values below the first cut and documents without an indexed value; the highest includes every value above the last cut. For multi-valued JSON, partitions use differences of nested range matches, so documents matching multiple ranges appear once. All partitions preserve static and dynamic filters. Each source rechecks its index schema before querying and fails if the partition field has become unsafe.

Pushed-down LIMIT, ORDER BY/top-N, grouping and reducers always use one split. Trino can still compute joins, unsupported expressions and exact DISTINCT from the parallel document sources. The normal cursor timeout, failure, exact BIGINT recovery and best-effort deletion behavior applies to each source. The pool bounds connections per worker; the split bound applies per query, not to the number of simultaneous queries. Configure Trino resource groups to bound overall analytical concurrency when sharing Redis with latency-sensitive traffic. Parallel scans do not provide snapshot isolation: keep the partition field stable during a scan, as moving a document across cut points during execution can cause omissions or repetitions.

SQL support

Tables and columns

Each Redis Query Engine index is a table in the `default` schema (see `redisearch.default-schema-name`). A table has these columns:

- One per indexed field, named by its attribute (the `AS` name for JSON paths).
- One per field that isn't indexed but appears in the first 10 documents returned by `FT.SEARCH <index> *`.
- For JSON indexes, a `$` column holding the whole document as JSON text.
- A hidden `key` column holding the document's Redis key. Hidden columns are left out of `SELECT *, so name it: SELECT key, * FROM beers`.

Table 4. Type mapping

Redis field type	Trino type
NUMERIC	DOUBLE
TAG, TEXT, GEO, GEOSHAPE, VECTOR	VARCHAR
Not indexed	VARCHAR

Columns created with `CREATE TABLE` or `ALTER TABLE ... ADD COLUMN` read back as the types they were created with instead; see [column types](#).

After `FT.CREATE` over existing keys, or `ALTER TABLE ... ADD COLUMN` (which runs `FT.ALTER`), Redis indexes the documents in the background. Until it finishes, queries on that index fail with `RESEARCH_INDEX_NOT_READY` (Index `beers` is still being built (42% indexed) ...) instead of returning incomplete results. Retry once `FT.INFO` reports `indexing` as `0`. `CREATE TABLE` creates its index with `SKIPINITIALSCAN`, so a new table can be queried right away.

Statements

Table 5. Supported statements

Statement	Notes
<code>SELECT</code>	All of Trino's query syntax, including joins (also across catalogs), subqueries, <code>WITH</code> , <code>UNION</code> , <code>DISTINCT</code> , <code>GROUP BY</code> , <code>ORDER BY</code> and window functions. Pushdown describes which parts run in Redis.
<code>SHOW SCHEMAS</code> , <code>SHOW TABLES</code> , <code>SHOW COLUMNS</code> , <code>DESCRIBE</code> , <code>SHOW CREATE TABLE</code> , <code>information_schema</code>	<code>SHOW TABLES</code> lists the indexes returned by <code>FT._LIST</code> .
<code>EXPLAIN</code>	The table scan shows the filters, limit and aggregations sent to Redis.

Statement	Notes
CREATE TABLE	Creates a hash index named after the table, with key prefix <code><table>:</code> . See column types . IF NOT EXISTS is supported.
CREATE TABLE AS SELECT	Columns get the same field types as in CREATE TABLE. A query with a column of another type fails before the table is created.
INSERT, INSERT ... SELECT	Hash indexes only: the connector writes hashes, which an index on JSON documents never sees, so it fails on a JSON index. Each row is a new hash with key <code><prefix><ULID></code> , where <code><prefix></code> is the index's first key prefix.
UPDATE	Hash indexes only. Setting a column to NULL removes the field from the hash.
DELETE	Deletes the matching keys, on hash and JSON indexes.
MERGE	Hash indexes only, except that WHEN MATCHED THEN DELETE also works on JSON indexes.
ALTER TABLE ... ADD COLUMN	Adds a field to the index with FT.ALTER. Columns are always added at the end. Queries on the table fail until Redis finishes reindexing its documents.
DROP TABLE	Drops the index and deletes its documents (FT.DROPINDEX ... DD).

These statements fail with a `This connector does not support ...` error:

- CREATE SCHEMA, DROP SCHEMA, ALTER SCHEMA
- CREATE OR REPLACE TABLE
- ALTER TABLE ... RENAME TO, DROP COLUMN, RENAME COLUMN, ALTER COLUMN ... SET DATA TYPE
- TRUNCATE
- COMMENT ON
- CREATE VIEW, CREATE MATERIALIZED VIEW
- Queries on table versions (FOR VERSION AS OF, FOR TIMESTAMP AS OF)

Writes also fail when fault-tolerant execution is enabled (`retry-policy` other than `NONE`).

Table 6. Column types in CREATE TABLE

Trino type	Index field	Stored in the hash as
BIGINT, INTEGER, SMALLINT, TINYINT, DOUBLE, REAL, DECIMAL	NUMERIC	The number, such as 42, 4.5 or 1.25
TIMESTAMP(3), TIMESTAMP(3) WITH TIME ZONE	NUMERIC	Epoch milliseconds
VARCHAR, CHAR, UUID	TAG	The string, with CHAR padded with spaces

Trino type	Index field	Stored in the hash as
BOOLEAN, DATE	TAG	true or false, and ISO dates such as 2024-01-02

Other types, such as ARRAY, MAP, ROW, VARBINARY and TIME, fail with **Unsupported column type**.

Redis keeps only each field's index type, which columns of several Trino types share. So **CREATE TABLE**, **CREATE TABLE AS SELECT** and **ALTER TABLE ... ADD COLUMN** also save the columns' Trino types, as JSON in the string key `__trino:columns:<table>`, such as `{"regionkey":"bigint","name":"varchar(25)"}`, and **DROP TABLE** deletes the key. Columns read back as those types in **DESCRIBE**, **SHOW CREATE TABLE**, `information_schema` and queries, and **INSERT** takes values of those types. **CREATE TABLE AS SELECT** keeps bounded types, so **SELECT 'a' AS id** makes a `varchar(1)` column.

The types in the key don't apply to:

- Indexes created outside Trino, and tables created by earlier versions of the connector, which didn't save types. Their columns read back as DOUBLE and VARCHAR.
- A field whose index type isn't the one the connector creates for the saved type, as after an index is dropped and created again outside Trino. It reads back as DOUBLE or VARCHAR.
- Any column, if the connector can't read the key: without read access to it in an ACL, or after Redis evicts it under an `allkeys- eviction policy`. **Creating a table needs write access to the key, such as `~__trino:` in an ACL rule.**

Redis returns a NUMERIC field loaded by name formatted as a double, rounded to 12 significant digits unless it's an integer. DOUBLE and DECIMAL scans use **LOAD *** on HASH tables to retain exact stored values. BIGINT and TIMESTAMP(3) scans load named fields and recover possibly rounded large values with exact HGETs. JSON scans use **DIALECT 3** to read numbers as stored. NUMERIC fields still hold doubles for queries and aggregations in Redis: see [Pushdown](#).

A value its column's type can't hold, such as 4.5 written by another client to a BIGINT column, fails the query with **Value '4.5' is not a valid bigint**. To read the table with the default types instead, delete the key (**DEL `__trino:columns:<table>`**); the first query after `redisearch.table-cache-refresh` seconds has the connector describe the table again, and the queries after it use the default types. The TAG fields aren't full-text TEXT fields. They're **CASESENSITIVE** and split values into tags on the ASCII unit separator (`\x1f`) instead of `,`, so Redis can prefilter `=` and **IN** on values containing commas, and doesn't return rows that only differ in case.

Pushdown

Redis evaluates these parts of a query. Trino evaluates everything else on the rows Redis returns.

- Filters on indexed fields, combined with **AND**:
 - `=`, **IN**, `<`, `<=`, `>`, `>=` and **BETWEEN** on NUMERIC fields of BIGINT, INTEGER, SMALLINT, TINYINT, DOUBLE, REAL, DECIMAL, TIMESTAMP(3) and TIMESTAMP(3) WITH TIME ZONE columns. NUMERIC fields hold doubles. Large BIGINT equality/IN values (including short inclusive ranges Trino merges from adjacent IN values) use widened numeric envelopes, retaining

exact residual verification in Trino. The envelopes include every matching signed 64-bit value and may include neighboring IDs. Large/unbounded unsafe BIGINT ranges, unsafe timestamp bounds, nullable domains and DECIMAL columns of more than 15 digits retain the existing fallback; no extra index is needed. Redis compares a REAL value as the double it parses from the text the connector writes, so `REAL '1.1'` as 1.1 rather than 1.100000023841858.

- `=` and `IN` on TAG fields, as a tag query, for VARCHAR, CHAR, BOOLEAN, DATE and UUID columns, whose values are compared as the connector writes them (`true`, `2024-01-02`, a lowercase UUID). On DATE columns, a range of up to 1,000 days, such as `BETWEEN`, is queried as the list of its dates. Redis splits stored values into tags on the field's `SEPARATOR` (by default `,` on hashes and none on JSON), trims whitespace around each tag, and ignores case unless the field is `CASESENSITIVE`. It returns the documents with a tag that matches the value. A `FILTER` step then compares the field's value as a string and keeps the documents equal to it, for example `FILTER 'exists(@style) && @style == "Wheat"'`. On JSON indexes it compares the first of the values at the field's path, and a boolean as `1` or `0`, which is how the connector reads them. Scans read JSON documents with `DIALECT 3`, whose values `FILTER` can't compare, so the connector keeps the equal documents itself, and Trino applies the `LIMIT`. A TAG field indexes a JSON boolean as the tag `true` or `false`, so on JSON indexes Trino evaluates `=` and `IN` with the values `1` and `0`. Trino evaluates the filter alone when a value contains the separator, starts or ends with whitespace, or has a control character other than whitespace, since a tag query can't match it.
- `=` and `IN` on TEXT fields, as a full-text query for the value's words. Redis returns the documents containing all of them, and the `FILTER` step keeps those equal to the value. Trino evaluates the filter alone when a value has only stop words, and for every TEXT field of an index created with its own `STOPWORDS`.

Trino keeps the equal rows itself, instead of a `FILTER`, when a value has a control character, for fields whose names aren't made of letters, digits and `_`, and for CHAR columns, whose values the connector pads with spaces.

Redis can't match a missing field or compare strings, so Trino evaluates `IS NULL`, `IS NOT NULL`, a condition combined with `OR ... IS NULL`, and `<>`, `<`, `>` and `BETWEEN` on VARCHAR columns.

On HASH fields with supported property names, `LIKE '%literal%'` without additional `%/_` wildcards, an `ESCAPE` clause or literal control characters uses an exact `contains` filter before loading the full projection. Trino evaluates other `LIKE` shapes and `JSON LIKE`. A Redis wildcard query matches single words of a TEXT field, ignores case on a TAG field without `CASESENSITIVE`, and leaves out documents once it has matched `MAXEXPANSIONS` (200 by default) distinct values.

- With `redisearch.aggregation-pushdown.enabled=true` (the default), `count()`, `min`, `max`, `sum` and `avg` on numeric columns, and `min` and `max` on `TIMESTAMP(3)` and `TIMESTAMP(3) WITH TIME ZONE` columns, with or without `GROUP BY`, as `FT.AGGREGATE` with `GROUPBY` and `REDUCE`. If any aggregate in a query can't be pushed down, such as `count(DISTINCT ...)` or `count(column)`, or Trino filters the rows itself, Trino computes all of them. `count(column)` skips nulls, but Redis's `COUNT` counts documents, so only `count()` is pushed down. `sum` and `avg` add up each document's value, or 0 for a document without one (`APPLY "case(exists(@x), @x, 0)"`), and count the values (`APPLY "exists(@x)"`); the connector returns null for a group with no values and

divides each average itself. On a sharded database, Redis's own `SUM` and `AVG` return null for a group when one shard has none of its values, and `AVG` divides by the number of documents. `case()` needs Redis 8.2 or later, or unstable features enabled on Redis 8.0 and Redis Stack 7.4; without it, and for columns whose names aren't made of letters, digits and `,`, the connector sends `SUM` and `AVG`, which are right on a single shard. Redis computes on the double values of `NUMERIC` fields, and formats the results and the `GROUP BY` keys rounded to 12 significant digits unless they're integers. So for each `DOUBLE` or `REAL` result and key, and the sum each avg is divided from, `APPLY` steps also return it as an integer mantissa and exponent, which Redis formats in full, and the connector reads the exact double. Over `RESP2` (`redisearch.resp2`, or a database without `RESP3`), the shards of a sharded database send these values to its coordinator already rounded, so Trino computes those aggregations. On a `BIGINT` column, results and keys are exact only for values and sums below 2^{53} . Trino groups `DECIMAL` columns itself, and `DOUBLE` and `REAL` columns whose names aren't made of letters, digits and `,`. Trino evaluates `HAVING` on the groups Redis returns. `GROUP BY` pushdown requires a known cached document count no greater than `redisearch.aggregation-group-limit`, since distinct cardinality is unknown. The count is an upper bound even for filtered queries. Cache staleness/index growth can still trigger a server error; failures are not replayed after delivering output.

- `sum` and `avg` of arithmetic on hash indexes, such as `sum(quantity * extendedprice): +, -, *, /` and unary `-` on `DOUBLE NUMERIC` columns, on integer `NUMERIC` columns cast to `DOUBLE`, and on `DOUBLE` constants. An `APPLY` step computes the arithmetic before `GROUPBY`, for the documents that have all of its columns (`APPLY "case(exists(@quantity) && exists(@extendedprice), (@quantity * @extendedprice), 0)"`), so it needs `case()` too. A scan that returns the arithmetic computes it in the connector, from the values as stored. Trino computes arithmetic on integers, which SQL fails on when it overflows, on `REAL` values, which Redis would compute as doubles, and `min` and `max` of arithmetic, since Trino orders nan above other doubles.
- `LIMIT`, unless Trino filters the rows itself. Eligible numeric `HASH TopN` uses Redis `SORTBY`. Single-column `TIMESTAMP(3) HASH TopN` with `NULLS LAST` and `LIMIT` up to 1000 scans only keys/order values and retains exact candidates before pipelined `HMGET` hydration of their projected fields. Other sort shapes remain in Trino. Timestamp selection checks the configured query timeout over the entire scan and deletes a live cursor on failure; it emits no partial candidates. As with ordinary scans, concurrent writes have no snapshot guarantee.

Known limitations

Queries known to return wrong results or fail are tracked as [open issues labeled bug](#).

Clients

JDBC Driver

The Trino JDBC driver allows users to access Trino from Java-based applications, and other non-Java applications running in a JVM.

Refer to the [Trino documentation](#) for setup instructions.

The following is an example of a JDBC URL used to create a connection to Redis SQL Trino:

```
jdbc:trino://example.net:8080/redisearch/default
```

Tableau

Refer to the [Tableau documentation](#) for setup instructions.

Trino CLI

Refer to the [Trino CLI documentation](#) for setup instructions.

Build

Run these commands to build the Trino connector for Redisearch from source (requires Java 25+):

```
git clone https://github.com/redis-field-engineering/redis-sql-trino.git
cd Redis SQL Trino
./mvnw clean package -DskipTests
```

Complete Walkthrough

Follow these step-by-step instructions to deploy a single-node Trino server on Ubuntu.

Install Java

Trino requires a 64-bit version of Java 25. It is recommended to use [Azul Zulu](#) as the JDK.

```
$ java -version
openjdk version "25.0.1" 2025-10-21 LTS
OpenJDK Runtime Environment Zulu25.30+17-CA (build 25.0.1+8-LTS)
OpenJDK 64-Bit Server VM Zulu25.30+17-CA (build 25.0.1+8-LTS, mixed mode, sharing)
```

Set up Trino

Download the Trino server tarball and unpack it.

```
wget https://repo1.maven.org/maven2/io/trino/trino-server/483/trino-server-483.tar.gz
mkdir /usr/lib/trino
tar xzvf trino-server-483.tar.gz --directory /usr/lib/trino --strip-components 1
```

Trino needs a data directory for storing logs, etc. It is recommended to create a data directory outside of the installation directory, which allows it to be easily preserved when upgrading Trino.

Create a data directory

```
mkdir -p /var/trino
```

Create an **etc** directory inside the installation directory to hold configuration files.

```
mkdir /usr/lib/trino/etc
```

Create a **node.properties** file.

/usr/lib/trino/etc/node.properties

```
node.environment=production
node.id=ffffffff-ffff-ffff-ffff-ffffffffffffff
node.data-dir=/var/trino
```

Create a JVM config file.

/usr/lib/trino/etc/jvm.config

```
-server
-Xmx16G
-XX:InitialRAMPercentage=80
-XX:MaxRAMPercentage=80
-XX:G1HeapRegionSize=32M
-XX:+ExplicitGCInvokesConcurrent
-XX:+ExitOnOutOfMemoryError
-XX:+HeapDumpOnOutOfMemoryError
-XX:-OmitStackTraceInFastThrow
-XX:ReservedCodeCacheSize=512M
-XX:PerMethodRecompilationCutoff=10000
-XX:PerBytecodeRecompilationCutoff=10000
-Djdk.attach.allowAttachSelf=true
-Djdk.nio.maxCachedBufferSize=2000000
--add-modules=jdk.incubator.vector
```

Create a config properties file.

/usr/lib/trino/etc/config.properties

```
coordinator=true
node-scheduler.include-coordinator=true
http-server.http.port=8080
discovery.uri=http://localhost:8080
```

Create a logging configuration file.

/usr/lib/trino/etc/log.properties

```
io.trino=INFO
```

Set up Redis SQL Trino

Download latest [release](#) and unzip without any directory structure under **plugin/redisearch**:

```
wget https://github.com/redis-field-engineering/redis-sql-trino/releases/download/v0.4.2/redis-sql-trino-0.4.2.zip
unzip -j redis-sql-trino-0.4.2.zip -d /usr/lib/trino/plugin/redisearch
```

Create a etc/catalog subdirectory:

```
mkdir /usr/lib/trino/etc/catalog
```

Create a Redisearch connector configuration file:

/usr/lib/trino/etc/catalog/redisearch.properties

```
connector.name=redisearch
redisearch.uri=redis://localhost:6379
```

Change and/or add [properties](#) as needed.

Start Trino

Start the Trino server:

```
/usr/lib/trino/bin/launcher run
```

Run Trino CLI

Download and run [trino-cli-483-executable.jar](#):

```
wget https://repo1.maven.org/maven2/io/trino/trino-cli/483/trino-cli-483-
executable.jar -O /usr/local/bin/trino
chmod +x /usr/local/bin/trino
trino --catalog redisearch --schema default
```

Run a SQL query:

```
trino:default> select * from mySearchIndex;
```